

Структуры данных в PHP.

Односвязанный список.

- Должно иметь значение (\$item).
- Должна быть ссылка на след. элемент (\$next).

конструктор заполняет структуру данных

```
class Node
{
    public function __construct(
        private string $item,
        private ?Node $next = null,
    ) {}

    public function getItem(): string
    {
        return $this->item;
    }

    public function getNext(): ?Node
    {
        return $this->next;
    }

    public function setNext(Node $node): void
    {
        $this->next = $node;
    }
}
```

Стэк - первый пришел, последним ушёл

```

class Stack extends Sequence
{
    public function __construct(
        private ?Node $last,
    ) {}

    public function put(string $item): void
    {
        $this->last = new Node($item, $this->last);
    }

    public function get(): ?string
    {
        if ($this->isEmpty()) {
            return null;
        }
        $item = $this->last->getItem();
        $this->last = $this->last->getNext();
        return $item;
    }

    protected function getFirst(): ?Node
    {
        return $this->last;
    }
}

```

Как это работает:

```

include 'class/Node.php';
include 'class/Stack.php';

$stack = new Stack();
$stack->put('John');
$stack->put('Alex');
$stack->put('Mike');

echo $stack->get() . "<br>\n"; // Mike
echo $stack->get() . "<br>\n"; // Alex
echo $stack->get() . "<br>\n"; // John
echo $stack->get() . "<br>\n"; // null

```

Для очереди и стэка сделаем общий класс.

```

abstract class Sequence
{
    abstract public function put(string $item): void;
    abstract public function get(): ?string;
    abstract protected function gethead(): ?Node;

    public function isEmpty(): bool
    {
        return null == $this->getFirst();
    }

    public function getList(): iterable
    {
        $curr = $this->getFirst();
        while($curr != null)
        {
            yield $curr->getItem();
            $curr = $curr->getNext();
        }
    }
}

```

В очереди должно быть два указателя: на первый и последний элементы

```

class Queue extends Sequence
{
    /** @var Node */
    private $head;

    /** @var Node */
    private $last;

    public function put(string $item): void
    {
        $node = new Node($item);
        if ($this->isEmpty())
        {
            $this->head = $node;
            $this->last = $node;
        } else {
            $this->last->setNext($node);
            $this->last = $node;
        }
    }

    public function get(): ?string
    {
        if ($this->isEmpty()) {
            return null;
        }
        $item = $this->head->getItem();
        $this->head = $this->head->getNext();
        return $item;
    }

    protected function getFirst(): ?Node
    {
        return $this->head;
    }
}

```

```
include 'class/Node.php';
include 'class/Sequence.php';
include 'class/Stack.php';
include 'class/Queue.php';

$stack = new Queue();
$stack->put('John 1');
$stack->put('Alex 2');
$stack->put('Mike 3');

foreach ($stack->getlist() as $item) {
    echo $item . "<br>\n";
}

echo "=====<br>\n";
echo $stack->get() . "<br>\n"; // John
echo $stack->get() . "<br>\n"; // Alex
echo $stack->get() . "<br>\n"; // Mike
```

Графы

Реализуем по способу "Матрица смежности":

```

class Graph
{
    // матрица смежности вершин:
    // $edges['A']['B'] = 12; // length
    // $edges['B']['A'] = 12;

    public function __construct(
        private array $edges = [],
    ) {}

    public function addNode(string $node)
    {
        $this->edges[$node] = [];
    }

    public function addEdge(string $node1, string $node2, string $length)
    {
        $this->edges[$node1][$node2] = $length;
        $this->edges[$node2][$node1] = $length;
    }

    public function getNodes(): iterable
    {
        foreach ($this->edges as $node => $edge)
        {
            yield $node;
        }
    }

    public function getEdges(): iterable
    {
        foreach ($this->edges[$node1] as $node2 => $length)
        {
            yield $node2 => $length;
        }
    }
}

```